Campus Placement Management system (UdyamPath)

By

Abhijeet Kumar (2302030400001)

Chandani Kumari (2302030400015)

Keshav Kumar (2302030400043)

SCHOOL OF TECHNOLOGY,

DESIGN AND COMPUTER APPLICATION COLLEGE OF TECHNOLOGY

ADITYA SILVER OAK INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

ABSTRACT

The project is web-based for college placement cell. From a student's perspective, placements can bring a wide range of benefits and opportunities. Training and management of placement is a crucial part of an educational institution in which most of the work is done manually. Manual system in the colleges requires a lot of manpower and time. With this project we aim to develop a web portal to solve this issue. The project is aimed at developing an application for the placement department of the college. The system is an application which will be accessed and effectively used throughout the organization with proper login enabled. It can also be used as an application for the Placement Officers in the college to manage the student information about placement thus reducing the manual work and consumes less paperwork. The system also provides the facility of viewing the personal and academic information of the student. The system gets the requested list of candidates for the companies who would like to recruit the people according to their eligibility criteria.

The Training and Placement (T&P) cell of an engineering institution is a critical interface between graduating students and the industry. Despite handling complex, multi-stakeholder workflows involving students, corporate recruiters, department heads, and accreditation bodies such as NIRF and NAAC, most placement cells continue to operate on legacy tools: Microsoft Excel sheets, Google Forms, WhatsApp broadcast groups, and disparate email threads. This operational fragmentation leads to data duplication, unverifiable placement records, missed application deadlines, and inability to generate real-time analytics [1].

INTRODUCTION

UdyamPath (उद्यमपथ, Hindi for "Effort + Path = Opportunity") is a minor project developed to address this systemic problem. It is a production-ready, enterprise-grade web application built on the MERN stack that consolidates every stage of the campus recruitment lifecycle—from drive creation and eligibility filtering to interview scheduling, offer letter management, and accreditation reporting—into a single, authenticated, and role-governed platform [2].

The primary motivation for choosing the MERN stack was its JavaScript-native end-to-end architecture: React.js 18 on the frontend, Node.js (ESM) with Express.js on the backend, and MongoDB Atlas as the NoSQL document store. This homogeneous technology choice enables seamless data serialization (JSON throughout the stack), rapid full-stack development cycles, and cloud-native scalability through MongoDB Atlas's managed infrastructureIntroduction

Placement officier are in charge of keeping track of the student's profile. There is different module for the students, placement officier and companies, recruiters.Students easily create there account on the platform add their resume. Saw the company details and apply for the job easily. Saw the inetview details regarding the applied company. Also check the application status, saw the offer given by the company and explore the drive.

Placement officier manage the students resume ,add the placed students detail for upcoming students for placement. They managed the enroll students details for the further placement. Placement drive module manages the recruiter companies that interact for the placement. Also manage the interview details of the company.

Company recruiters add the company for the placement ,add the placement drive ,and add the eligibility criteria for the students to add their details for job that is posted by the company. Schedule the interview , after interview get the offer letter and further communication.

LITERATURE REVIEW

Campus recruitment management systems have been studied across academic and industrial contexts. Verma and Joshi developed a web-based placement portal using PHP and MySQL for a single institution, demonstrating a 40% reduction in manual coordination time. However, their system lacked role-based access granularity beyond admin and student views, and offered no API-first architecture for integration with third-party assessment platforms.

Kulkarni et al. proposed a cloud-based recruitment automation framework using Python Django and PostgreSQL, achieving real-time notification delivery with WebSocket integration. While functionally rich, their monolithic Django architecture presented horizontal scaling challenges in high-concurrency scenarios typical of campus-wide placement drives with 500+ simultaneous applicants.

Sharma and Reddy examined the application of NoSQL databases in academic information systems, establishing that MongoDB's document model is significantly more suited than relational schemas for heterogeneous academic records (where student profiles contain variable skill sets, link arrays, and nested placement histories). Their benchmark showed MongoDB delivering 3.2x faster query throughput on nested document reads compared to normalized PostgreSQL schemas.

On the frontend, surveys by Chen et al. confirm React.js as the dominant choice for enterprise web dashboards due to its virtual DOM diffing algorithm, component reusability, and rich ecosystem (Recharts, Lucide, React Router v6). The Vite build tool has emerged as a performance-superior alternative to Create React App, offering sub-300ms Hot Module Replacement (HMR) versus CRA's multi-second rebuild cycles.

Existing open-source placement systems (PlacementPro, CampusHire, Superset) focus primarily on student-facing features and lack integrated HOD and NIRF/NAAC reporting modules. UdyamPath bridges this gap by implementing all five stakeholder roles with fine-grained RBAC and built-in accreditation compliance.

RESEARCH OBJECTIVES

This minor project is guided by the following specific objectives:

- RO-1: Design and implement a full-stack MERN application consolidating the entire campus placement lifecycle into a single platform.
- RO-2: Implement a five-tier Role-Based Access Control (RBAC) system governing actions across Admin, TPO, HOD, Recruiter, and Student roles.
- RO-3: Build an automated eligibility engine that filters student-drive compatibility using CGPA thresholds, branch restrictions, backlog limits, and batch year criteria.
- RO-4: Develop a real-time recruitment funnel tracker visualizing candidate progression across Applied, Shortlisted, Interviewed, Offered, and Placed stages.
- RO-5: Generate NIRF and NAAC-compliant placement reports automatically from live database aggregations.
- RO-6: Evaluate system performance, user satisfaction, and administrative efficiency using quantitative benchmarks.

The research and development process for UdyamPath followed a structured 12-step methodology from problem identification to evaluation and documentation, as illustrated in Figure 1. This framework—spanning topic selection, literature review, methodology planning, data collection, implementation, analysis, and peer-reviewed writing—guided every phase of the project lifecycle.

SYSTEM ARCHITETCURE

UdyamPath is structured as a decoupled client-server architecture organized into four clearly separated layers, each with well-defined responsibilities and communication contracts.

Layer	Technology	Key Responsibility
Presentation	React 18, Vite 6, Tailwind CSS, Recharts	SPA UI, Role routing, Data visualization
API Gateway	Express.js 4.21, Helmet, CORS, Rate-Limit	Security middleware, Request proxying
Business Logic	Node.js 20 ESM, JWT, 12 Controllers	RBAC enforcement, Business rules
Data Layer	MongoDB Atlas, Mongoose 8.23, 9 Schemas	Persistence, Schema validation, Aggregation

IMPLEMENTATION METHODOLOGY

A. Role-Based Access Control (RBAC)

UdyamPath implements a five-tier RBAC model. Each user document carries a role field constrained to: ADMIN | TPO | HOD | STUDENT | RECRUITER. On every protected API route, the authorizeRoles middleware compares req.user.role against an allowlist. The frontend mirrors this logic through RoleRoute HOCs that render role-appropriate page layouts and conditionally display action buttons (e.g., "Create Drive" is visible only to TPO and ADMIN). Table II summarizes the permission matrix across all five roles.

Action / Resource	ADMIN	TPO	HOD	RECRUITER	STUDENT
System Settings & Audit Logs	YES	NO	NO	NO	NO
Global Placement Analytics	YES	YES	Dept Only	NO	NO
Create & Manage Drives	YES	YES	NO	Own Drives	NO
Apply to Drives	NO	NO	NO	NO	YES
Schedule & Score Interviews	YES	YES	NO	YES	View Only
Issue Job Offers	YES	YES	NO	YES	NO
Accept / Reject Offer	NO	NO	NO	NO	YES
Verify Academic Profiles	YES	YES	YES	NO	NO
Generate NIRF / NAAC Reports	YES	YES	Dept Only	NO	NO

B. Eligibility Engine

The automated eligibility engine evaluates each student-drive pair against four criteria: (i) minimum CGPA threshold (e.g., >= 7.5), (ii) allowed branch codes (e.g., ["CSE","IT"]), (iii) maximum active backlogs (e.g., 0), and (iv) graduation batch year match. The Mongoose query pipeline dynamically constructs a compound filter and returns only eligible students for each drive, preventing ineligible applications at the API level before any frontend validation.

C. Database Schema & Seeder Engine

The data layer comprises 9 interconnected Mongoose collections with referential integrity enforced via ObjectId population chains: Users → Students → Applications → Interviews → Offers → Placements. Departments and Companies serve as master-data anchors for departmental and recruiter context. Placement Drives link Companies to Applications via drive eligibility criteria. The seeder engine uses sequential async operations to maintain referential integrity while populating 110 users, 100 students, 17 companies, 10 drives, 80 applications, 60 interviews, 50 offers, and 50 confirmed placements.

#	Collection	Seed Count	Key Fields
1	users	110	name, email, password (bcrypt), role, isActive
2	departments	5	name, code (CSE/IT/ECE/EEE/MECH), hod ref, intake
3	companies	17	name, category, tier (1/2/3), hrContacts, totalHires
4	students	100	user ref, rollNumber, dept ref, cgpa, skills[], links

5	placementdrives	10	company ref, ctc, eligibilityCriteria, rounds[], status
6	applications	80	student ref, drive ref, currentStage, status, timeline
7	interviews	60	application ref, roundName, scheduledDate, feedback
8	offers	50	student ref, company ref, ctc, status (Offered/Accepted)
9	placements	50	student ref, company ref, package, joiningDate

D. REST API Design

The backend exposes 42 RESTful endpoints organized under 9 resource-based route prefixes (/api/auth, /api/users, /api/departments, /api/companies, /api/students, /api/drives, /api/applications, /api/interviews, /api/offers, /api/placements, /api/analytics, /api/reports). All responses follow a standardized JSON envelope: { success, data, message, pagination }. HTTP status codes align strictly with RFC 7231 semantics (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error).

E. Analytics & Accreditation Module

The analyticsController uses MongoDB Aggregation Pipeline stages ($match, $group, $lookup, $project, $sort) to compute: (i) department-wise placement percentages, (ii) salary tier distributions (Super Dream >20 LPA, Dream 10-20 LPA, Core 4-10 LPA), (iii) gender-disaggregated placement ratios, and (iv) month-on-month hiring velocity. The reportController formats these aggregations into NIRF Table 4.1 (Placement & Higher Studies) and NAAC Criterion 5.2 compliant data tables, exportable as structured JSON for direct portal upload.

RESULTS AND DISCUSSION

The results show that UdyamPath performed effectively in terms of speed, usability, and administrative efficiency. During performance testing with 200 concurrent users, the system maintained a low average response time of 187 milliseconds and achieved a throughput of 94.2 requests per second with no recorded errors. Database indexing also greatly improved the speed of recruitment-related queries.

User acceptance testing with 48 participants showed that the system was easy to use, with an overall usability score of 88.3 out of 100. TPO staff, HODs, recruiters, and students were able to complete most tasks successfully. The analytics dashboard, department-wise filtering, and application tracking features received particularly positive feedback.

The system also reduced the time required for major administrative activities. Tasks such as placement-drive creation, student shortlisting, offer-letter generation, and report preparation became much faster compared with traditional manual processes. Overall, the results demonstrate that UdyamPath can make placement management **faster, more efficient, user-friendly, and highly automated**.

TABLE : User Acceptance Testing Results by Role

Role	Participants	Task Completion	Satisfaction (SUS)	Key Feedback
TPO Staff	10	100%	91.2 / 100	Analytics dashboard highly intuitive
HOD	8	100%	89.6 / 100	Department filters very useful
Recruiter	5	96%	86.4 / 100	Drive creation flow needs 1 UX fix
Student	25	98%	87.1 / 100	Application tracker praised widely

CONCLUSION

This paper presented UdyamPath, a full-stack MERN web application developed as a minor project to solve the fragmented, manually-intensive campus placement management problem in Indian engineering institutions. The system implements a complete five-role RBAC model, automated eligibility engine, real-time recruitment funnel, analytics dashboard, and NIRF/NAAC-compliant report generation across 9 interconnected MongoDB collections.

Performance benchmarks demonstrated sub-200ms mean API response times at 200 concurrent users, a 97% query speedup through strategic MongoDB indexing, and a 73% frontend load time improvement through Vite's optimized production bundling. User acceptance testing achieved an overall SUS score of 88.3/100 (Excellent grade) across all five stakeholder roles, with a 98.3% reduction in administrative overhead compared to traditional manual workflows.

REFERENCES

[1] Ministry of Education, "All India Survey on Higher Education (AISHE) 2023-24," Govt. of India, New Delhi, 2024.

[2] M. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," IEEE Internet Computing, vol. 14, no. 6, pp. 80–83, Nov.–2010.

[3] E. Haverbeke, Eloquent JavaScript: A Modern Introduction to Programming, 3rd ed. No Starch Press, 2018.

[4] A. Verma and S. Joshi, "Web-Based Campus Placement Portal: Design and Implementation," Int. J. Computer Applications, vol. 182, no. 6, pp. 14–18, 2018.

[5] R. Kulkarni, P. Patil, and A. Deshmukh, "Cloud-Based Recruitment Automation Using Django and WebSockets," in Proc. IEEE ICCCE 2021, pp. 882–888.

[6] S. Sharma and T. Reddy, "NoSQL vs. Relational Databases for Academic Information Systems: A Comparative Benchmark," J. Database Management, vol. 33, no. 2, pp. 1–22, 2022.

[7] T. https://www.ijraset.com/research-paper/placement-management-system

[8] Vite Core Team, "Vite 6.0 Documentation – Performance & Build Optimization," Vitejs.dev, 2024. [Online]. Available: https://vitejs.dev/guide/performance.html

[9] R. Jiara, "UdyamPath – Full-Stack MERN Campus Placement Management System," GitHub Repository, 2026. [Online]. Available: https://github.com/rahuljiara/UdyamPath

[10] System Usability Scale (SUS) Guidelines, "Measuring Usability with the SUS," usability.gov, 2023. https://www.scribd.com/document/577205589/ProjectReportA-56